Lecture 3. Errors, Overfitting, Model Selection, and Evaluation in Classification

Data mining, prof. Biljana Tojtovska Ribarski
FCSE, Skopje

Contents

1 Errors, Overfitting, and Model Evaluation in Classification

1.1 Introduction

A classification model is built to make predictions on records that were not used during training. Because of this, the central question in classification is not whether a model fits the training data well, but whether it generalizes well to unseen data.

In the previous lecture we asked whether the predictors and splits were meaningful. Now we ask whether a tree that looks reasonable on the training data will still work well on unseen data.

In practice, a model may achieve very small error on the training set and still perform poorly on new observations. This gap between apparent performance and true predictive performance is one of the main reasons why model evaluation and model selection are essential topics in classification.

Decision trees provide a particularly clear setting for studying these ideas. A tree can be made very small and simple, or it can be grown into a highly detailed structure with many nodes. As the tree becomes more complex, training error typically decreases. However, this does not guarantee that test error will also decrease.

This chapter studies the basic types of classification errors, explains how overfitting arises, and presents principled ways to select, simplify, and evaluate models.

1.2 Classification Errors

When evaluating a classifier, several notions of error must be distinguished.

Definition: Training error

The *training error* (also called *apparent error*) is the error committed by the classifier on the same training records that were used to build the model.

Definition: Test error

The *test error* is the error committed by the classifier on a separate test set that was not used during model training.

Definition: Generalization error

The *generalization error* is the expected prediction error of the model on new records drawn from the same underlying distribution as the training data.

Training error is easy to compute, but it is usually optimistic. Test error is more informative, provided the test set has not influenced model building. Generalization error is the ideal quantity of interest, but it is typically unknown and must be estimated indirectly.

Remark: Why these three errors differ?

A classifier is optimized to perform well on the training set. Therefore training error is often lower than test error. Generalization error is broader still: it refers not only to one fixed test set, but to future data sampled from the same population.

Insight: Test error is an estimate, not the target quantity itself

Test error is an estimate of generalization error, not the generalization error itself.
They are often reasonably close when:

- the test set is large enough;
- the test set is representative of the target population;
- the test data come from the same distribution as future data;
- the test set is not reused during tuning or repeated model revisions;
- there is no leakage, that is, no unintended flow of test or future information into the training process.

They can differ substantially when the test set is small or biased, when future data come from a different population, when the same test set is inspected many times during tuning, or when the data are very noisy or highly variable.

There are different ways to define the error that we will measure. We can use the misclassification error, as we will do for simplicity for the rest of the chapter. But we can also use some of the known measures as accuracy, F1 score, precision etc, which we will introduce in the next sections.

Example: A simple interpretation

Suppose a classifier is trained on 200 student records and evaluated on 100 unseen student records.

- If it misclassifies 10 of the 200 training cases, the training error is $10/200 = 0.05$.
- If it misclassifies 18 of the 100 unseen cases, the test error is $18/100 = 0.18$.

The difference between these values suggests that the classifier may not generalize as well as the training error alone would indicate.

1.3 A Decision-Tree Example of Increasing Complexity

To understand overfitting concretely, it is useful to imagine a two-class problem in which one class forms a concentrated cluster and the other class is more dispersed. A decision tree can approximate the decision boundary by recursively partitioning the feature space into axis-aligned regions. (See the graph in the slides.)

A small tree creates a coarse partition. A larger tree creates a more detailed partition. As additional nodes are added, the tree can fit increasingly fine-grained irregularities in the training data.

The important observation is that training error usually decreases as more nodes are added, but this decrease does not by itself tell us whether the model is improving in a meaningful predictive sense.

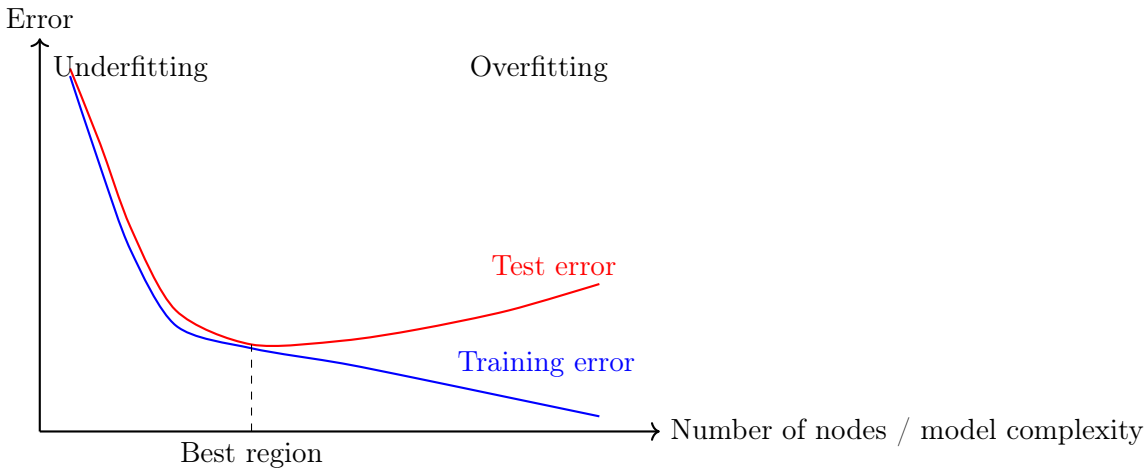

Figure 1: Typical relationship between model complexity and classification error. Training error usually decreases as complexity grows, but test error may decrease first and then increase.

1.4 Underfitting and Overfitting

Definition: Underfitting

A model is said to *underfit* when it is too simple to capture the important structure in the data. In this case, both training error and test error are typically large.

Definition: Overfitting

A model is said to *overfit* when it is too complex and starts fitting accidental patterns, noise, or peculiarities of the training set. In this case, training error may be small while test error is large.

Underfitting and overfitting are opposite failures:

- An underfit model has not learned enough.
- An overfit model has learned too much from the particular training sample, including aspects that do not generalize.

Insight: Central practical principle

As model complexity increases, training error often decreases monotonically. Test error does not have to do so. This is the essence of overfitting.

Example: Two decision trees

Consider a decision tree with 4 nodes and another with 50 nodes built on the same training set.

- The smaller tree may give a rough but meaningful partition of the feature space.
- The larger tree may carve out many tiny regions that fit unusual training points.

If the larger tree mainly captures noise, its training error will be lower, but its test performance may be worse.

1.5 Effect of Training Set Size

One of the simplest and most important observations in machine learning is that larger training sets often reduce overfitting.

When more training data are available:

- the model sees a broader sample of the population;
- accidental peculiarities have less influence;
- the difference between training error and test error often becomes smaller.

A highly complex model may still overfit, but the same model complexity is usually less dangerous when the dataset is much larger.

Definition: Data-size effect on overfitting

At a fixed model complexity, increasing the size of the training set often reduces the gap between training error and test error.

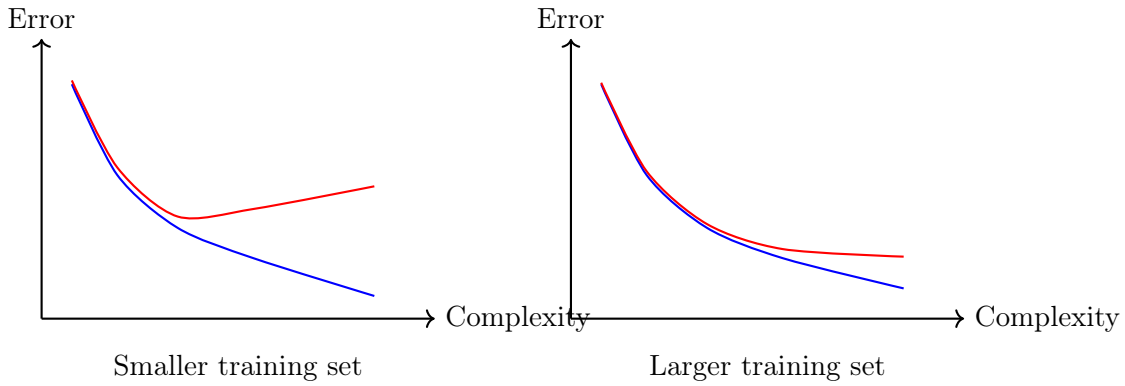

Figure 2: A schematic illustration of how increasing the training set size can reduce the gap between training and test error at a given model complexity.

Remark: Interpretation

More data do not automatically solve all modeling problems, but they often make it harder for the model to memorize accidental patterns. Still, if the dataset and the the predictors

are not suitable for the classification problem, increasing the dataset will not improve the predictive power of the model.

1.6 Why Overfitting Happens

Two major reasons for overfitting are especially important in classification:

- limited training size;
- high model complexity.

A further subtle issue is the *multiple-comparison effect*, which appears whenever a learning algorithm searches through many candidate model components and keeps whichever one appears best.

Definition: Model complexity

Model complexity refers to the expressive flexibility of the model. For decision trees, complexity increases with quantities such as depth, number of internal nodes, number of leaves, and the fineness of the induced partition.

1.6.1 The Multiple-Comparison Effect

Suppose many candidate variables, thresholds, or test conditions are examined. Even if none of them carries real signal, some may look useful by chance on the training set.

This means that an algorithm can overfit not only because it is flexible, but also because it is given many opportunities to discover spurious improvements.

Example: Stock prediction analogy

Suppose a person makes random guesses about whether the stock market will go up or down over 10 days. The probability of getting at least 8 out of 10 guesses correct by chance is not very large for one individual. But if 50 people each guess randomly, the probability that at least one of them appears remarkably successful becomes much higher (almost 1). If we then select the "best" person, we may wrongly believe we found skill, when in fact we only found chance.

Insight: Why this matters for learning algorithms?

Many learning algorithms use a greedy rule of the form:

$$M' = M \cup \gamma,$$

where γ is a candidate component to add to the model, such as a split in a decision tree. If γ is chosen from a large set of alternatives, some irrelevant component may seem helpful purely by chance and may be added, increasing overfitting risk.

Example: Noisy variables

Suppose a two-dimensional classification task can already be solved well using variables X and Y. If we now add 100 completely noisy variables and allow a decision tree to search over all of them, some of those noisy variables may look useful on the training data. This can lead to a more complex tree and worse test performance.

Warning: A practical lesson

The presence of many irrelevant or noisy variables increases the risk that a greedy algorithm will discover accidental improvements and build an unnecessarily large model.

Remark: A practical workflow when many noisy variables are present

If there are many noisy variables, a decision tree may split on them by chance and overfit. Therefore, one should control complexity with pruning and penalties, and choose tree size using validation or cross-validation rather than training error alone.
A practical interpretation is:

- **Before building:** remove variables that are obviously irrelevant, duplicated, leaky, or meaningless for prediction, when domain knowledge and data inspection justify this.
- **While building:** use pre-pruning or stopping rules so that weak accidental improvements do not create unnecessary nodes.
- **After building:** use post-pruning to remove subtrees that do not improve estimated generalization.
- **For selection:** compare candidate tree sizes using a validation set or cross-validation instead of relying on training error alone.

1.7 Why Training Error Is Not Enough?

Training error usually decreases when a model becomes more flexible. For this reason, it is an *optimistic* measure of performance. It tells us how well the model has fit the seen data, not how well it will perform on unseen records.

Definition: Optimistic estimate

An estimate of model performance is called *optimistic* if it tends to report better performance than the model actually achieves on new data.

Remark: Training error as an optimistic estimate

Resubstitution error, that is, error measured on the training data, is often optimistic because the same data were used both to build and to evaluate the model.

This is why model selection requires methods for estimating generalization error rather than simply minimizing training error.

1.8 Model Selection

Definition: Model selection

Model selection is the process of choosing among alternative models or model sizes during training, with the goal of obtaining good generalization rather than merely low training error.

The purpose of model selection is to avoid choosing a model that is more complex than necessary. In decision trees, this often means determining whether a node should be split, whether a subtree should be retained, or whether the tree should be pruned.

Three general approaches are especially important:

1. using a validation set;
2. incorporating model complexity directly;

3. estimating statistical bounds on error.

Algorithm: What quantity is actually compared during pruning?

When deciding whether to keep a split, keep a subtree, or prune it away, what we actually compare is an *estimated generalization error*. This estimate can be obtained in several ways.

1. **Validation-set estimate.** Build candidate trees, or candidate pruned versions of a tree, on a training subset and compare them on a validation subset. Then keep the version with lower validation error or better validation performance.
2. **Complexity-penalized estimate.** Use a criterion of the form

   $$\text{training error} + \text{penalty for complexity}.$$

   For decision trees, this becomes the pessimistic error estimate, where larger trees or trees with more leaves receive a larger penalty.
3. **Statistical upper bound on error.** Instead of only adding a direct complexity penalty, adjust the observed error upward to reflect uncertainty, especially when leaves are small. Then compare the parent node with its children, or compare a full subtree with its pruned version, using those upper bounds.

Thus, pruning decisions should not be based on training error alone; they should be based on whichever estimate of out-of-sample performance the chosen procedure uses.

1.8.1 Model Selection Using a Validation Set

A standard strategy is to divide the available training data into two parts:

- a **training subset**, used to build the model;
- a **validation subset**, used to estimate generalization error and choose model complexity.

Definition: Validation set

A *validation set* is a subset of the available training data that is reserved for model selection, hyperparameter tuning, or early stopping. It is different from the final test set.

This approach is conceptually simple and often effective. Its drawback is that it reduces the amount of data available for fitting the model itself.

Warning: Validation set versus test set

The validation set is used during model building. The test set is not. If the test set influences model design, it no longer provides an honest assessment of performance.

1.8.2 Incorporating Model Complexity

Another approach is to evaluate a model by combining its training fit with a penalty for complexity. This reflects the principle known as Occam's Razor.

Definition: Occam's Razor in model selection

If two models achieve similar predictive performance, the simpler model is generally preferred because it is less likely to have fitted accidental structure.

A generic penalized criterion can be written as

$$\text{Estimated Generalization Error} = \text{Training Error} + \lambda \cdot \text{Complexity(Model)},$$

where λ is a trade-off parameter.

The intuition is simple:

- good fit is desirable;
- complexity is risky;
- model selection balances the two.

Insight: Interpretation of the penalty term

A complexity penalty does not assume that all complex models are bad. Instead, it says that a complex model should justify its complexity by improving enough to offset its added risk.

1.9 Pessimistic Error Estimate for Decision Trees

For decision trees, one way to incorporate complexity is to use a *pessimistic error estimate*. The idea is to start from the observed training error and then add a penalty that increases with the number of leaf nodes.

Definition: Pessimistic error estimate

For a decision tree T with leaf nodes $t_1, \ldots, t_k$, a pessimistic error estimate has the form

$$e_{\text{gen}}(T) = \frac{\sum_{i=1}^{k} \left[e(t_i) + \Omega(t_i) \right]}{\sum_{i=1}^{k} n(t_i)} = \frac{e(T) + \Omega(T)}{N(T)},$$

where:

- $e(t_i)$ is the number of misclassified training records in leaf t_i,
- $n(t_i)$ is the number of training records reaching leaf t_i,
- $e(T)$ is the total number of misclassified training records in the tree,
- $\Omega(T)$ is a penalty term reflecting model complexity,
- $N(T)$ is the total number of training records.

The role of the penalty is to discourage unnecessary splitting.

Example: Interpretation of a penalty of 0.5

If each additional leaf contributes a penalty of 0.5, then splitting a node is worthwhile only if the improvement in training classification is strong enough to compensate for that added complexity. In this sense, the penalty acts as a built-in resistance against growing the tree too eagerly.

Remark: What should influence the penalty parameter Ω?

The penalty parameter Ω should mainly reflect the risk of overfitting and the goal of the task.

- If the dataset is small, noisy, high-dimensional, or contains many irrelevant variables, then accidental splits are more likely, so a stronger penalty or stricter pruning is often reasonable.
- If the dataset is much larger and the signal is strong, then each split is supported by more evidence, so one can often allow a weaker penalty.

Why does validation help? Because the penalty parameter should be chosen by asking: which value gives the best out-of-sample performance? In practice, one usually builds candidate trees for several penalty strengths, or several pruning settings, and compares them on a validation set or with cross-validation.
The chosen value is the one that gives the best estimated generalization according to the *relevant metric*. For example, in a medical screening task, one may care more about sensitivity/recall than raw accuracy, so the preferred pruning strength is the one that performs best on sensitivity or recall, not necessarily the one with the smallest overall error. (See the lecture about evaluation metrics.)

1.9.1 A Small Worked Example

Suppose two candidate trees are built from the same 24 training instances (see presentation):

- Tree T_L makes 4 training errors and has 7 leaves;
- Tree T_R makes 6 training errors and has 4 leaves.

With penalty parameter $\Omega = 1$, the pessimistic estimates are

$$e_{\text{gen}}(T_L) = \frac{4+7}{24} = \frac{11}{24} \approx 0.458, \qquad e_{\text{gen}}(T_R) = \frac{6+4}{24} = \frac{10}{24} \approx 0.417.$$

Although T_L has lower training error, T_R has better penalized error because it is simpler.

Remark: Important lesson

A model with lower training error is not necessarily preferable once complexity is taken into account.

==========

1.10 Pruning Decision Trees

Pruning is the process of simplifying a decision tree so that it generalizes better.

There are two broad approaches:

- pre-pruning;
- post-pruning.

1.10.1 Pre-Pruning

Definition: Pre-pruning

Pre-pruning stops tree growth early, before the tree becomes fully developed.

Typical stopping rules include:

- stop if all instances at a node belong to the same class;
- stop if all attribute values are the same;
- stop if the number of instances is below a user-defined threshold;
- stop if the class distribution appears independent of the candidate features;
- stop if impurity does not improve enough;

- stop if estimated generalization error no longer improves.

Remark: Advantage and risk of pre-pruning

Pre-pruning can prevent overfitting and reduce computation, but if used too aggressively it may stop the tree before important structure is discovered.

1.10.2 Post-Pruning

Definition: Post-pruning

Post-pruning first grows the tree more fully and then removes subtrees whose presence does not improve generalization.

Two common ideas are:

- **subtree replacement**: replace a subtree by a single leaf if this improves the estimated generalization error;
- **subtree raising**: replace a subtree with one of its branches when this yields a simpler and better tree.

Example: Subtree replacement

Suppose a node contains 30 training records with majority class `Yes`. If replacing the whole subtree by a single leaf gives a pessimistic error estimate of 10.5/30, while keeping the split gives 11/30, then the subtree should be pruned.

Warning: Do not confuse lower training error with better tree

A subtree may slightly reduce training error and still be undesirable if it increases estimated generalization error.

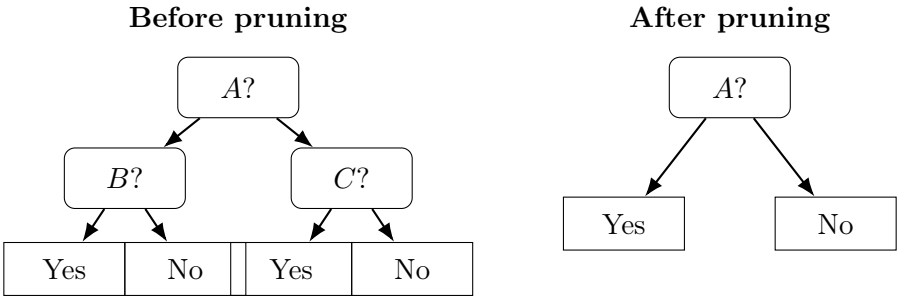

Figure 3: Illustration of post-pruning by subtree replacement. A more complex subtree may be replaced by a simpler leaf structure if estimated generalization improves. See specific example in the slides.

1.11 Model Evaluation

Model selection and model evaluation are related but not identical.

Definition: Model evaluation

Model evaluation is the process of estimating how well a classifier performs on previously unseen data.

The main goal is to estimate predictive performance honestly.

1.11.1 Holdout Method

Definition: Holdout method

In the *holdout method*, the available data are partitioned into a training set and a test set. The classifier is trained on the former and evaluated on the latter.

For example, one may reserve 70% of the data for training and 30% for testing.

Remark: Repeated holdout

Instead of performing one random split only once, one may repeat the holdout procedure several times with different random splits. This is called *repeated holdout* or *random subsampling*.

The holdout method is simple and easy to implement, but its estimate may be unstable if the dataset is small or if the particular split is not representative.

1.11.2 Cross-Validation

Definition: k-fold cross-validation

In *k-fold cross-validation*, the data are partitioned into k disjoint subsets (folds). The model is trained k times, each time using $k-1$ folds for training and the remaining fold for testing.

The final estimate is usually the average of the k test errors.

Special cases include:

- **leave-one-out cross-validation**: $k = n$, so each observation serves as a test point once;
- **repeated cross-validation**: perform cross-validation multiple times with different fold assignments;
- **stratified cross-validation**: preserve class proportions in each fold (important for imbalanced classes).

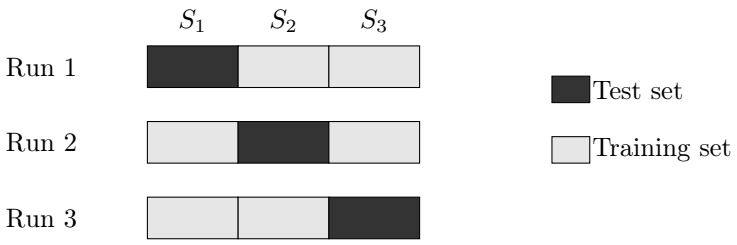

Figure 4: Example of 3-fold cross-validation. Each subset serves once as the test set and twice as part of the training set.

Definition: Stratified cross-validation

Stratified cross-validation preserves approximately the same class proportions in each fold as in the full dataset.

This is especially important when:

- the classes are imbalanced;

- the sample size is small.

Definition: Nested cross-validation

Nested cross-validation uses an inner cross-validation loop for model selection and an outer cross-validation loop for final performance evaluation.

Nested cross-validation is particularly important when model selection is substantial, because it prevents optimistic bias from tuning and testing on effectively the same data.

Warning: Cross-validation misuse

If hyperparameters are chosen using the same folds that are then reported as final performance, the resulting estimate may be too optimistic. Nested cross-validation is designed to avoid this.

Warning: Preprocessing leakage inside evaluation.

If scaling, imputation, encoding, or feature selection is performed on the full dataset before cross-validation, the evaluation may become optimistically biased. Such preprocessing steps must be fitted using only the training portion of each split or fold, and then applied to the corresponding validation or test portion.

Remark: Default settings are only generic starting points

Default tree settings are chosen by a software library for broad usability, not for your exact sample size, noise level, class imbalance, or real prediction goal. Therefore, leaving them unchanged can easily produce an overgrown or misleading tree.
In practice, the algorithm and the data scientist have different roles:

- the **algorithm** provides a rule for searching candidate splits and building candidate trees;
- the **data scientist** must decide how complexity will be controlled, which evaluation metric matters, and how model selection and performance estimation will be carried out.

A good practical workflow is to treat the defaults only as a baseline, then adapt stopping rules, minimum leaf size, maximum depth, pruning strength, and evaluation metric to the problem using validation or cross-validation. This is especially important for small or imbalanced datasets, where one should control depth and leaf size more carefully and should not rely only on training accuracy.

1.12 Practical Do's and Don'ts

Do's

- Distinguish clearly among training, validation, and test performance.
- Expect training error to be optimistic.
- Use validation sets or cross-validation for model selection.
- Prefer simpler models when their performance is comparable.
- Use pruning or other regularization to control decision-tree complexity.
- Use stratified cross-validation for imbalanced data.
- Use nested cross-validation when both tuning and evaluation are important.

- Treat default tree settings as baselines, not as final problem-specific decisions.
- Choose the evaluation metric to match the task; for example, in screening problems sensitivity/recall may matter more than raw accuracy.

Don'ts

- Do not choose the model solely because it has the smallest training error.
- Do not confuse a validation set with a test set.
- Do not report performance from a test set that influenced model design.
- Do not interpret very deep trees as automatically better.
- Do not ignore the role of noisy variables and multiple comparisons.
- Do not assume that a single random split is always reliable.
- Do not assume that a library default is automatically appropriate for your sample size, class imbalance, noise level, or decision goal.

Insight: A robust guiding principle

A good classifier is not the one that explains the training set most perfectly. It is the one that predicts unseen data most reliably.

1.13 Summary

Training error, test error, and generalization error are distinct concepts, and confusing them leads to poor conclusions. A classifier may perform very well on the training set while still generalizing poorly.

Overfitting occurs when a model becomes too complex and begins to fit accidental patterns in the training data. Underfitting occurs when the model is too simple to capture the relevant structure. In decision trees, these ideas are especially visible because increasing the number of nodes almost always lowers training error, while test error may eventually increase.

Overfitting is encouraged by limited training size, excessive model complexity, and multiple comparisons among many candidate variables or model components. Good model selection therefore requires more than minimizing training error.

Three main strategies were exist:

- using a validation set;
- penalizing model complexity;
- using statistical bounds on error (it is not discussed here).

For decision trees, these ideas lead naturally to pre-pruning and post-pruning. Finally, honest performance assessment requires proper evaluation procedures such as holdout, cross-validation, stratified cross-validation, repeated cross-validation, and nested cross-validation.

In practice, one should also remember that default library settings are only generic starting points. The data scientist must choose a suitable complexity control strategy, a suitable evaluation protocol, and a suitable performance metric for the actual problem.

The main message is simple but fundamental:

The goal of classification is not to fit the observed data as closely as possible, but to predict new data as accurately as possible.

1.14 A Small Practice Example with Data

The following dataset contains two informative variables, X_1 and X_2, and a binary class label. The pattern is intentionally simple, but the points near the middle of the table make it possible for a larger tree to start fitting minor irregularities.

ID	X_1	X_2	Class
1	2.0	1.0	0
2	2.5	2.0	0
3	3.0	1.5	0
4	3.2	3.0	0
5	4.0	2.5	0
6	4.5	4.0	0
7	5.0	4.2	0
8	5.2	4.8	1
9	5.5	5.2	1
10	6.0	5.5	1
11	6.5	6.0	1
12	7.0	6.5	1
13	7.2	7.0	1
14	8.0	6.8	1
15	8.5	7.5	1
16	9.0	8.0	1

Table 1: A small dataset for practicing overfitting, pruning, and model evaluation.

Example: Practice task

Use this dataset in a decision-tree package of your choice.

1. Fit an unpruned tree using only X_1 and X_2. Record the tree depth, number of leaves, training error, and 4-fold or 5-fold stratified cross-validation error.
2. Now create additional noisy variables $Z_1, \ldots, Z_{20}$ independently from a Uniform$(0, 1)$ distribution and add them to the dataset.
3. Fit an unpruned tree again using X_1, X_2, and all noisy variables. Compare its depth, number of leaves, training error, and cross-validation error with the previous tree.
4. Next, tune at least one complexity-control parameter, such as maximum depth, minimum leaf size, or pruning strength, using validation or cross-validation.
5. Decide which model you would select:
   - if the goal is overall accuracy;
   - if the goal is a screening task in which missing class 1 is more serious, so recall/sensitivity matters more. The evaluation metrics are defined in the next lecture.

With this example you should observe several core ideas from the chapter: training error often decreases with complexity, noisy variables can lead to more accidental splits, and model choice should be based on estimated generalization rather than training fit alone.

1.15 Exercises

1. Explain the difference between training error, test error, and generalization error.

2. Why can training error be a poor estimate of predictive performance?

3. Give a conceptual explanation of underfitting and overfitting.

4. Why can increasing the number of nodes in a decision tree reduce training error but increase test error?

5. Explain why adding many noisy variables can lead to overfitting.

6. What is the multiple-comparison effect, and why is it relevant for greedy learning algorithms?

7. Why is a complexity penalty useful in model selection?

8. What is the difference between pre-pruning and post-pruning?

9. Why is stratified cross-validation useful for imbalanced datasets?

10. Why is nested cross-validation preferable when extensive model tuning is performed?

11. A fully grown tree has better training accuracy than a pruned tree, but worse validation recall on a small medical screening dataset. Which tree should be preferred, and why is the answer not determined by training accuracy alone?

12. Suppose a team repeatedly evaluates many different tree depths and pruning settings on the same test set until the reported result looks good. Explain why the final reported test error becomes optimistic, and describe a correct workflow using either train/validation/test splitting or nested cross-validation.

13. Compare two scenarios for choosing the penalty parameter Ω:

    (a) a small, noisy, high-dimensional biomedical dataset with many irrelevant variables;

    (b) a large industrial sensor dataset with strong signal and much more data.

    Explain in which case a stronger penalty is more reasonable, how validation or cross-validation should be used to tune Ω, and why the relevant evaluation metric may differ across tasks.